

Тема: Методы

Для решения задач данного раздела только можно использовать лекционный материал, который включает данную тему, либо идет ранее.

Массив можно задавать как вводом через консоль, так и инициализацией. Если используется инициализация, то заранее подготовьте несколько примеров (один активный, остальные закомментированы).

Необходимо наличие дружеского интерфейса (ввод/вывод в консоль должен сопровождаться пояснениями).

Теоретический материал

Метод — это именованный фрагмент кода *внутри класса*, который можно вызвать из другого метода того же класса или из метода другого класса.

Методы в языке Java могут быть *статическими* и *обычными*. *Статический метод*, по существу, — это функция, объявленная в пространстве имен класса. Статический общедоступный метод можно вызвать из любого места программы без необходимости создания экземпляра класса. *Обычный метод* привязан к классу и может быть вызван только через экземпляр класса.

Общее определение методов выглядит следующим образом:

1	[модификаторы] тип_возвращаемого_значения название_метода ([параметры]){
2	// тело метода
3	}

Модификаторы и параметры необязательны.

После названия метода, внутри круглых скобок, указываются типы и названия параметров через запятую. Если метод не принимает параметров, то указываются только круглые скобки. Название параметра является *локальной переменной*. Эта переменная создается при вызове метода, а после выхода из метода удаляется. Таким образом, локальная переменная видна только внутри метода, и ее значение между вызовами не сохраняется. Если название локальной переменной совпадает с названием глобальной переменной, то все операции будут производиться с локальной переменной, а значение глобальной не изменится. Чтобы в этом случае обратиться к глобальной переменной, необходимо перед ее названием указать название класса и оператор «точка»: `MyClass.x`

При вызове метода указывается название метода, после которого, внутри круглых скобок, передаются значения. Если метод не принимает аргументов, то указываются только круглые скобки. Количество и тип параметров в определении метода должны совпадать с количеством и типом аргументов при вызове. Если тип не совпадает, то производится попытка автоматически преобразовать типы без потерь данных. Если это невозможно, то будет выведено сообщение об ошибке. Переданные значения присваиваются переменным, расположенным в той же позиции в определении метода. Так, при вызове метода `sum(10, 20)` (формат `int sum (int x, int y)`) переменной `x` будет присвоено значение 10, а переменной `y` значение 20.

В метод передается копия значения переменной. То есть изменение значения внутри метода не затронет значения исходной переменной. Передача элементарных типов и неизменяемых объектов (например, строк) никогда не изменяет значения исходных переменных. Однако если передается изменяемый объект, то копируется лишь ссылка, а не объект.

Генерация случайных чисел

Чтобы сгенерировать случайное число Java предоставляет класс `Math`, доступный в пакете `java.util`. Этот класс содержит статичный метод `Math.random()`, предназначенный для генерации случайных чисел типа `double`.

Метод `random()` возвращает положительное число большее или равное 0,0 и меньшее 1,0:

```
double a = Math.random();
```

Для генерации случайных чисел в диапазоне [min; max) возможно использовать

```
(Math.random() * ((max - min) + 1)) + min
```

Класс `java.util.Random` можно применять для генерации случайных чисел различных типов: `int`, `float`, `double`, `long` и `boolean`. Для этого сначала создайте экземпляр класса `Random`, а затем вызовите один из методов генератора случайных значений, например: `nextInt()`, `nextDouble()` или `nextLong()`.

Класс `java.util.Random` реализует линейный конгруэнтный генератор (LCG). Он отличается быстротой работы. Но при этом он не подходит для использования в режиме реального времени. Например, для генерации уникального идентификатора сессии на сервере, в научных экспериментах, криптографии, лотереях и розыгрышах.

Индивидуальные задания

Название класса **Main**

Общее техническое задание: Структурное программирование и методы в Java

1. **Цель работы.** Отработать навык **декомпозиции** (разделения) сложной задачи на простые составляющие части. Научиться проектировать интерфейсы методов: определять входные параметры и типы возвращаемых значений, а также обеспечивать их корректное взаимодействие.

2. **Ограничения и правила.** При реализации заданий необходимо строго придерживаться следующих правил:

- **Запрет на ООП:** Не создавать пользовательские классы (кроме основного класса с `main`). Не использовать поля класса (статические переменные) для передачи данных между методами.
- **Типы данных:** Разрешено использовать только примитивные типы (`int`, `double`, `boolean`, `char`) и одномерные массивы этих типов.
- **Принцип «Один метод — одна задача»:** Каждый метод должен выполнять строго определенную логическую операцию.
- **Декомпозиция:** В каждом задании должно быть реализовано минимум **два метода** (помимо `main`), которые вызывают друг друга или используются совместно для решения задачи.
- **Чистота методов:** Метод-вычислитель не должен напрямую выводить данные в консоль (через `System.out.println`), если это не является его основной задачей. Он должен **возвращать результат** в `main` или другой метод.

3. **Требования к реализации.** Для каждого задания в классе должны присутствовать:

1. **Вспомогательный метод (Level 1):** Решает элементарную подзадачу (например, проверка одного числа, расчет одной величины, сравнение двух элементов).
2. **Управляющий метод (Level 2):** Обрабатывает массив или последовательность данных, используя вызовы вспомогательного метода в цикле или логических условиях.
3. **Метод `main`:** Служит «точкой сборки». Здесь инициализируются входные данные (массивы, константы), вызывается управляющий метод и выводится финальный результат на экран.

4. **Критерии оценки**

- **Корректность сигнатур:** Правильно ли выбраны типы аргументов и возвращаемого значения.
- **Отсутствие дублирования:** Не дублируется ли логика вспомогательного метода внутри управляющего.
- **Обработка краевых случаев:** Как программа ведет себя с пустым массивом, нулевыми значениями или отрицательными числами (где это критично).
- **Читаемость:** Понятные названия методов (глаголы, отражающие суть: `calculate...`, `is...`, `find...`) и переменных.

Задание 1. Детектор финансовых аномалий. Написать программу, которая помечает подозрительно крупные транзакции относительно среднего чека.

- **Метод 1:** Принимает массив и вычисляет среднее арифметическое (только для положительных чисел).
- **Метод 2:** Принимает массив и «порог риска» (коэффициент). Выводит индексы тех транзакций, которые превышают среднее значение в N раз.
- **На выходе:** Список индексов «подозрительных» операций.

Задание 2. Сглаживание показаний датчика. Написать программу, которая убирает точечные помехи из массива данных (фильтр скользящего среднего).

- **Метод 1:** Принимает три числа (предыдущее, текущее, следующее) и возвращает их среднее арифметическое.
- **Метод 2:** Обходит массив и заменяет каждый элемент (кроме крайних) результатом работы первого метода.
- **На выходе:** Массив с «плавными» переходами значений.

Задание 3. Анализ производственных циклов. Написать программу, которая находит самую длинную серию дней с одинаковым объемом выпуска продукции.

- **Метод 1:** Сравнивает два целых числа и возвращает `true`, если они равны.
- **Метод 2:** Находит в массиве длину самой протяженной последовательности идущих подряд равных элементов.
- **На выходе:** Одно число — максимальная длина «плато».

Задание 4. Многофакторный валидатор пароля. Написать программу, которая оценивает сложность пароля (массива `char`).

- **Метод 1:** Принимает символ и возвращает его тип: 1 (буква), 2 (цифра), 3 (спецсимвол).
- **Метод 2:** Проверяет массив. Пароль считается надежным, если его длина больше 8 и в нем присутствуют минимум две разные категории символов.
- **На выходе:** `true` или `false`.

Задание 5. Поиск локальных максимумов (Трейдинг). Написать программу, которая фиксирует «пики» стоимости актива.

- **Метод 1:** Принимает три числа и возвращает `true`, если центральное число строго больше левого и правого соседей.
- **Метод 2:** Подсчитывает количество таких пиков во всем массиве цен.
- **На выходе:** Количество найденных экстремумов.

Задание 6. Формирование идеального склада. Написать программу, которая объединяет отчеты двух складов, выбирая по каждой позиции максимальный остаток.

- **Метод 1:** Сравнивает два числа и возвращает большее из них.
- **Метод 2:** Принимает два массива (склад А и склад Б) одинаковой длины и создает третий массив, заполняя его лучшими показателями через вызов первого метода.
- **На выходе:** Результирующий массив остатков.

Задание 7. Проверка на критический брак. Написать программу, которая бракует партию, если две бракованные детали лежат в коробке рядом.

- **Метод 1:** Принимает число и возвращает `true`, если оно меньше 0 (маркер брака).
- **Метод 2:** Проверяет массив и возвращает `false` (партия не годна), как только находит два идущих подряд бракованных элемента.
- **На выходе:** Вердикт: «Партия принята» или «Партия отклонена».

Задание 8. Оптимизация навигационных точек. Написать программу, которая фильтрует маршрут, удаляя «пустые» перемещения.

- **Метод 1:** Считает расстояние между двумя точками по координатам.
- **Метод 2:** Выводит в консоль только те точки маршрута, расстояние от которых до предыдущей точки больше 0.1 км.
- **На выходе:** Список координат «значимых» остановок.

Задание 9. Дифференцированный налог. Написать программу, которая рассчитывает чистую прибыль с учетом прогрессивной шкалы.

- **Метод 1:** Принимает доход филиала. Если доход больше 150000, налог составляет 20%, если меньше — 13%. Возвращает сумму после налога.
- **Метод 2:** Суммирует «чистые» доходы всех филиалов из массива.
- **На выходе:** Итоговая сумма прибыли компании.

Задание 10. Поиск скрытых дубликатов. Написать программу, которая находит ошибки в списке ID пользователей.

- **Метод 1:** Принимает массив и число, возвращает количество его повторов в этом массиве.
- **Метод 2:** Проходит по массиву и, используя первый метод, выводит на экран только те ID, которые встретились более одного раза.
- **На выходе:** Список повторяющихся элементов (без учета того, сколько раз они повторились).

Задание 11. Анализ пульса (Спорт). Написать программу, которая определяет, находился ли атлет в целевой зоне тренировки.

- **Метод 1:** Принимает значение пульса и диапазон (min, max). Возвращает `true`, если пульс внутри диапазона.
- **Метод 2:** Проверяет массив ударов сердца и вычисляет процент времени (от общего количества замеров), проведенного в целевой зоне.
- **На выходе:** Число (процент) времени эффективной тренировки.

Задание 12. Сверка остатков (Торговля). Написать программу, которая находит расхождения между фактическим наличием товара и данными в базе.

- **Метод 1:** Сравнивает два числа и возвращает их разность (по модулю).
- **Метод 2:** Принимает два массива (план и факт) и выводит индексы товаров, где расхождение составило более 5 единиц.
- **На выходе:** Список порядковых номеров проблемных позиций.

Задание 13. Детектор «разрядки» (Энергетика). Написать программу, которая ищет момент самого резкого падения напряжения в сети.

- **Метод 1:** Принимает два числа (предыдущее и текущее) и возвращает величину падения (разность, если она положительна, иначе 0).
- **Метод 2:** Находит в массиве пару соседних элементов с максимальным падением и возвращает индекс второго элемента.
- **На выходе:** Номер замера, на котором зафиксирован «скачок» вниз.

Задание 14. Фильтр уникальных символов (Тексты). Написать программу, которая очищает строку (массив `char`) от повторяющихся знаков препинания.

- **Метод 1:** Принимает два символа и возвращает `true`, если оба они являются знаками препинания (например, `!`, `?`, `.`).
- **Метод 2:** Проходит по массиву и выводит символы в консоль, пропуская текущий символ, если он совпадает с предыдущим и оба являются знаками препинания.
- **На выходе:** Массив (или вывод в консоль) без дублей знаков (например, `"!!!"` превращается в `"!"`).

Задание 15. Рейтинг популярности (Маркетинг). Написать программу, которая определяет «хиты продаж» (товары, чьи продажи выше среднего по категории).

- **Метод 1:** Рассчитывает среднее значение массива (исключая нулевые продажи).
- **Метод 2:** Создает новый массив (той же длины), где на месте «хитов» стоит `1`, а на месте остальных — `0`.
- **На выходе:** Массив-маска, показывающий успешные товары.

Задание 16. Оценка зачета (Образование). Написать программу, которая выставляет «зачет», если средний балл выше 4.0 и нет ни одной «двойки».

- **Метод 1:** Принимает массив оценок и возвращает `true`, если в нем есть хотя бы одна двойка (число 2).
- **Метод 2:** Рассчитывает средний балл и, используя первый метод, выносит вердикт («Зачтено» / «Не зачтено»).
- **На выходе:** Текстовый результат.

Задание 17. Синхронизация потоков (IT). Написать программу, которая проверяет, приходят ли пакеты данных строго вовремя.

- **Метод 1:** Принимает время прихода пакета и ожидаемое время. Возвращает `true`, если задержка не более 10 мс.
- **Метод 2:** Принимает массив фактического времени и массив планового. Возвращает общее количество «опоздавших» пакетов.
- **На выходе:** Целое число — количество сбоев.

Задание 18. Обрезка краев (Фотообработка). Написать программу, которая удаляет «черные поля» (нули) по краям снимка (массива).

- **Метод 1:** Принимает число и возвращает `true`, если это «черный пиксель» (0).
- **Метод 2:** Находит первый и последний индексы элементов, которые НЕ являются нулями.
- **На выходе:** Два индекса — границы полезных данных в массиве.

Задание 19. Расчет зарплаты с бонусом (HR). Написать программу, которая начисляет премию только тем сотрудникам, кто выполнил норму.

- **Метод 1:** Принимает сумму продаж и норму. Если сумма выше нормы, возвращает 15% от продаж, иначе 0.
- **Метод 2:** Суммирует базовый оклад (фиксированный) и бонусы, рассчитанные первым методом для каждого сотрудника.
- **На выходе:** Итоговая сумма выплат всему отделу.

Задание 20. Детектор симметрии (Геометрия). Написать программу, которая проверяет, является ли узор (массив чисел) зеркальным.

- **Метод 1:** Сравнивает два числа на равенство.
- **Метод 2:** Проверяет весь массив, сравнивая элементы i и $n-1-i$. Возвращает `false`, если найдено хотя бы одно несовпадение.
- **На выходе:** Сообщение: «Симметрично» или «Асимметрично».

Задание 21. Детектор «тишины» (Аудиоаналитика). Написать программу, которая находит участки аудиозаписи без звука (нулевая амплитуда).

- **Метод 1:** Принимает число и возвращает `true`, если оно равно 0 (тишина).
- **Метод 2:** Подсчитывает, сколько в массиве «пауз» длиной более 3-х элементов подряд.
- **На выходе:** Количество найденных длинных пауз.

Задание 22. Пересечение интересов (Социальные сети). Написать программу, которая находит общих друзей у двух пользователей.

- **Метод 1:** Принимает массив и число, возвращает `true`, если число есть в массиве.
- **Метод 2:** Принимает два массива ID друзей. Выводит в консоль те ID из первого массива, которые есть и во втором (используя первый метод).
- **На выходе:** Список совпадающих ID.

Задание 23. Контроль «разбега» цен (Агрегатор). Написать программу, которая сравнивает цены конкурентов и помечает слишком дорогие позиции.

- **Метод 1:** Принимает две цены и возвращает процент разницы между ними.
- **Метод 2:** Принимает массив ваших цен и массив цен конкурента. Выводит индексы товаров, где ваша цена выше конкурентной более чем на 20%.
- **На выходе:** Порядковые номера товаров, требующих пересмотра цены.

Задание 24. Проверка «шахматного» порядка (Логика). Написать программу, которая проверяет, чередуются ли в массиве четные и нечетные числа.

- **Метод 1:** Принимает два числа и возвращает `true`, если одно из них четное, а другое — нет.
- **Метод 2:** Проверяет весь массив. Если хотя бы одна пара соседних элементов имеет одинаковую четность, возвращает `false`.
- **На выходе:** Вердикт: «Порядок соблюден» или «Нарушение чередования».

Задание 25. Сжатие данных (Архиватор). Написать программу, которая заменяет идущие подряд одинаковые значения на их количество и само значение (упрощенно).

- **Метод 1:** Сравнивает текущий элемент со следующим и возвращает `true`, если они разные (конец серии).
- **Метод 2:** Проходит по массиву и выводит в консоль длину текущей серии одинаковых чисел, как только серия прерывается.
- **На выходе:** Текстовый отчет (например, для 1, 1, 1, 2, 2 выведет "3 раза по 1, 2 раза по 2").

Задание 26. Поиск «безопасного» пути (Робототехника). Написать программу, которая находит в массиве препятствий самый широкий свободный проход.

- **Метод 1:** Принимает число и возвращает `true`, если это препятствие (например, 1).
- **Метод 2:** Считает длину каждой серии нулей (свободный путь) и возвращает максимальную найденную длину.
- **На выходе:** Размер самого широкого прохода.

Задание 27. Рейтинг по дельте (Геймдев). Написать программу, которая поощряет игроков за самый большой прогресс в очках между уровнями.

- **Метод 1:** Вычитает из текущего результата предыдущий и возвращает разницу.
- **Метод 2:** Находит в массиве очков за 10 уровней максимальный положительный «скачок» (результат первого метода).
- **На выходе:** Индекс уровня, на котором игрок показал лучший прогресс.

Задание 28. Валидатор банковской карты (FinTech). Написать программу, которая проверяет контрольную сумму цифр (упрощенный алгоритм).

- **Метод 1:** Принимает число и возвращает его удвоенное значение; если результат больше 9, вычитает из него 9.
- **Метод 2:** Суммирует элементы массива цифр карты, применяя первый метод к каждому второму элементу.
- **На выходе:** Общая сумма (для проверки кратности 10).

Задание 29. Поиск пропущенного ID (Администрирование). Написать программу, которая находит «дыру» в идущих по порядку идентификаторах.

- **Метод 1:** Сравнивает два числа и возвращает `true`, если между ними есть разрыв (например, 5 и 7).
- **Метод 2:** Проходит по отсортированному массиву ID и выводит первое число, которое должно было идти следующим, но отсутствует.
- **На выходе:** Значение пропущенного ID.

Задание 30. Температурная инверсия (Метеорология). Написать программу, которая считает, сколько раз за день температура падала ниже ночного минимума.

- **Метод 1:** Принимает текущую температуру и эталон. Возвращает 1, если текущая ниже, иначе 0.
- **Метод 2:** Принимает массив дневных температур и значение ночного минимума. Суммирует результаты первого метода.
- **На выходе:** Общее количество случаев аномального похолодания.